# *Machine Learning A*

*2026-2027*

## Home Assignment 6

**Christian Igel** **Sadegh Talebi** **Yevgeny Seldin**

Department of Computer Science
University of Copenhagen

The deadline for this assignment is **22 October 2026, 22:00**. You must submit your *individual* solution electronically via the Absalon home page.

A solution consists of:

- A PDF file with detailed answers to the questions, which may include graphs and tables if needed. Do *not* include your full source code in the PDF file. You may include short, selected code snippets if you are asked to do so or if they help improve the readability or clarity of your answers.
- Your solution source code, with comments about the major steps involved in each question (see below). If your solution consists of multiple source files, submit them together in a .zip file; if there is only one source file, it may be submitted directly without zipping. Source code must be submitted in the original file format, not as PDF. The programming language of the course is Python.
- IMPORTANT: Do NOT zip the PDF file, since zipped files cannot be opened in *SpeedGrader*. Zipped PDF submissions will not be graded.
- Your PDF report should be self-sufficient. That is, it should be possible to grade your submission without opening the source-code submission. In particular, all requested outputs, such as plots, tables, and numerical values, must be generated and included in the PDF report. We do not guarantee that the source-code submission will be opened during grading.
- Your code should be structured such that there is one main file (or one main file per question) that we can run to reproduce all the results presented in your report. This main file can, if you like, call other files with functions, classes, etc.

- Handwritten solutions will not be accepted. Please use the provided LaTeX template to write your report.
- You must adhere to the notations used in the task and in the lectures. If you need to use additional notations, ensure they are clearly and precisely defined. Undefined notations will result in point deduction.

# 1 Conceptual Questions `([optional] 0 points) [Sadegh, W5 Monday]`

When deriving generalization bounds, we made some basic assumptions about data.

(i) What are these assumptions?

(ii) Briefly explain what happens when these assumptions break.

# 2 Function Basis `(22 points) [Christian, W6 Monday]`

This assignment uses artificial data, however, it is inspired by a current real-world data science task. Load the data in the file `functions.npy` using `np.load("functions.npy")`.

Each line $k$ in the data file contains the measurements of a function $f^{(k)}$ regularly sampled at the equidistant points $\mathcal{X} = \{i \cdot \Delta \mid i = 0, \ldots, 100\}$ with $\Delta = 0.06$ (but the concrete value of $\Delta$ does not really matter here). We know that each function is defined by a weighted sum of $F$ unknown functions $g_i$ (*basis functions*) and that the measurements are slightly corrupted by additive mean-free noise. The number $F$ of underlying functions as well as the weights $\boldsymbol{\alpha}^{(k)} \in \mathbb{R}^F$ for each sampled function are unknown. Formally, the data in line $k$ are generated by

$$f^{(k)}(x) = \sum_{i=1}^{F} \alpha_i^{(k)} g_i(x) + \epsilon \tag{1}$$

where $\epsilon$ represents mean-free i.i.d. Gaussian noise.

1. Plot the mean function (i.e., the mean of the measurements of the functions $f^{(k)}$ at the points $\mathcal{X}$).
2. Each basis function is defined by its function values for $x \in \mathcal{X}$. Estimate basis functions from the data. This can be done by a method introduced in the course.

(a) How many functions $g_i$ – in addition to the mean function – are needed to explain 95% of the variance of the observed data? We assume this value to be $F$. Make a plot with the number of basis functions and the explained variance as abscissa and ordinate, respectively.

(b) Plot the estimated $F$ functions over $\mathcal{X}$.

3. Plot the first function $f^{(1)}$ over $\mathcal{X}$ and its reconstruction using (1) with the estimated basis functions.

4. Based on your results, give an estimate for the variance of the noise $\epsilon$. Briefly explain your reasoning.

**Comment:** The basis functions are denoted by $g_1, \ldots, g_F$, not $g_0, \ldots, g_{F-1}$. In Matplotlib you can change the axis tick labels like this: `plt.xticks( range(0, N+tickstep, tickstep), range(1, N+tickstep+1, tickstep) )`

# 3 Sleep Well Revisited (10 points) [Christian, W6 Monday]

Sleep is one of the most fundamental physiological processes, and abnormal sleeping patterns are associated with poor health. They may, for example, indicate brain- & heart diseases, obesity and diabetes. During sleep our brain goes through a series of changes between different *sleep stages*, which are characterized by specific brain and body activity patterns. *Sleep staging* refers to the process of mapping these transitions over a night of sleep. This is of fundamental importance in sleep medicine, because the sleep patterns combined with other variables provide the basis for diagnosing many sleep related disorders (Kales and Rechtschaffen, 1968, Iber and AASM, 2007). The stages can be determined by measuring the neuronal activity in the cerebral cortex (via electroencephalography, EEG), eye movements (via electrooculography, EOG), and/or the activity of facial muscles (via electromyography, EMG) in a *polysomnography* (PSG) study. The classification into stages is done manually. This is a difficult and time-consuming process, in which expert clinicians inspect and segment the typically 8–24 hours long multi-channel signals. Contiguous, fixed-length intervals of 30 seconds are considered, and each of these *segments* is classified individually. Algorithmic sleep staging aims at automating this process. The state-of-the-art in algorithmic sleep staging is marked by deep neural networks, which can be highly accurate and robust, even compared to human performance, see the recent work by Perslev et al. (2019) and references therein.

This assignment considers algorithmic sleep staging. The data is based on a single EEG channel from the Sleep-EDF-15 data set (Kemp et al., 2000,

Goldberger et al., 2000). The input is given by an intermediate representation from the U-Time neural network architecture (Perslev et al., 2019), the targets are sleep stages. We created a training and test set, the inputs and the corresponding labels can be found in `X_train.csv` and `y_train.csv` and `X_test.csv` and `y_test.csv`, respectively. Download and extract the data from `https://github.com/christian-igel/ML/tree/main/data/Sleep`.

### 3.1 Principal component analysis

Perform a principal component analysis of the training data `X_train.csv`. Plot the eigenspectrum (see the plot on slide 30 of the *PCA* slides for an example). How many components are necessary to "explain 90 % of the variance"? Visualize the data by a scatter plot of the data projected on the first two principal components. Use different colors for the different classes in the plot.

*Deliverables:* description of software used; plot of the eigenspectrum; number of components necessary to explain 90 % of variance; scatter plot of the data projected on the first two principal components with different colors indicating the 5 different classes

## 4 Differentiable programming (24 points)

`[Christian, W4 Friday]`

This task should make you more familiar with PyTorch, automatic differentiation, and iterative gradient-based optimization. These are basics of deep learning, but differential programming is useful in scientific modelling beyond neural networks.

Consider the notebook `https://github.com/christian-igel/ML/tree/main/notebooks/MLA/Gradientexample.ipynb`. made available in the context of the lecture on linear classification. Your task is to migrate it to PyTorch and use autograd instead of specifying the gradient of the function explicitly. Use the Torch optimizers. Reproduce the plot created by `Gradient example.ipynb`.

*Deliverables:* show the modified parts of the code in the report and explain how and why your migration works

**Hints:**

- In the beginning, you need to define the trainable parameters of the function you want to optimize. By setting `requires_grad=True` you inform autograd that it should keep track of the gradient of a tensor. The initialization of

the parameters should not be taken into account by autograd. Thus, you have to tell autograd to look away when doing the initialization:

```
r = 1.
x = torch.tensor([0.9*r],requires_grad=True)
y = torch.tensor([0.8*r],requires_grad=True)
```

- For a tensor with a single element, you can get the value in standard Python by calling `torch.Tensor.item()`.
- For steepest descent use `torch.optim.SGD([x, y], lr=eta)`.
- Do not forget to reset the gradients of your optimizer in the optimization loop.
- A plot of the optimization steps should look like in the example not using PyTorch – so you can check whether your solution is correct.

# 5 Generalization Bounds for Sample Splitting (28 points) [Sadegh, W7 Monday]

In this question you will analyze one possible approach to the question of how to split a dataset $S$ into training and test sets, $S^{\texttt{train}}$ and $S^{\texttt{test}}$. As we have already discussed, overly small test sets lead to unreliable loss estimates, whereas overly large test sets leave too little data for training, thus producing poor prediction models. The optimal trade-off depends on the data and the prediction model. So can we let the data speak for itself? We will give it a try.

We want to find a good balance between the sizes of $S^{\texttt{train}}$ and $S^{\texttt{test}}$. We consider $m$ possible splits $\{(S_1^{\texttt{train}}, S_1^{\texttt{test}}), \ldots, (S_m^{\texttt{train}}, S_m^{\texttt{test}})\}$, where the sizes of the test sets are $n_1, \ldots, n_m$, correspondingly. For example, it could be $(10\%, 90\%), (20\%, 80\%), \ldots, (90\%, 10\%)$ splits or anything else with a reasonable coverage of the possible options. We train $m$ prediction models $\hat{h}_1^*, \ldots, \hat{h}_m^*$, where $\hat{h}_i^*$ is trained on $S_i^{\texttt{train}}$. We calculate the test loss of the $i$-th model on the $i$-th test set $\hat{L}(\hat{h}_i^*, S_i^{\texttt{test}})$. Derive a bound on $L(\hat{h}_i^*)$ in terms of $\hat{L}(\hat{h}_i^*, S_i^{\texttt{test}})$ and $n_i$ that holds for all $\hat{h}_i^*$ simultaneously with probability at least $1 - \delta$.

*Comment: No theorem from the lecture notes applies directly to this setting, because they all have a fixed sample size $n$, whereas here the sample sizes $n_1, \ldots, n_m$ vary. You have to provide a complete derivation.*

# 6 Fusion via Generalization Bounds (16 points)

[Sadegh, W7 Monday]

You are a team leader and you have 3 excellent data scientists in your team. You have received data for binary classification from a new client. The data consists of 2000 annotated i.i.d. samples. You gave the data to your team members and asked them to provide you a classifier. The team members are all proud individualists and they decided that each of them will work on the data independently of the rest. And they did the following:

1. The first scientist has split the data into 1000 training and 1000 test points, did all the training on the training set and then tested the final output on the test set and got a test error $\hat{L}(\hat{h}_1^*, S_1^{\texttt{test}}) = 0.1$.
2. The second scientist has split the data into 1500 training and 500 test points, did all the training on the training set and then tested the final output on the test set and got a test error $\hat{L}(\hat{h}_2^*, S_2^{\texttt{test}}) = 0.08$.
3. The third scientist has split the data into 1750 training and 250 test points, did all the training on the training set and then tested the final output on the test set and got a test error $\hat{L}(\hat{h}_3^*, S_3^{\texttt{test}}) = 0.075$.

All three have reported their classifiers and test errors to you. Now you have to decide which of the three classifiers you will provide to your client. Explain how you will pick the classifier and provide a generalization bound for the classifier that you will return to the client. The generalization bound should hold with probability at least 95%.

# 7 Efficient Use of Data ([optional] 0 points)

[Sadegh, W7 Monday]

So far, given a data set $S$, in all our theoretical results we used part of the data for training prediction rules and another part for validating them. This way some data are only used for training and some data are only used for validation. But put attention that if we would have trained a prediction rule on the validation set and validated it on the training set, we would have also gotten an unbiased estimate of the loss. (Remember: "it's not about how you call it, it's about how you use it"!). So could we use the data more efficiently?

The approach of using part of the data for training and part for validation, and then reverting the roles, somewhat resembles cross-validation. But a big word of warning would be in place here. Even though the standard cross-validation

technique is widely used, it is a heuristic, and if it is used to validate too many prediction rules, it is prone to overfitting, in exactly the same way as the standard validation technique is prone to overfitting, unless generalization bounds are used to control the overfitting.

What you will do next is inspired by cross-validation, but it is different from the standard cross-validation approach.

So, we have a data set $S$ of size $n$ (assume that $n$ is even). We split the data set into two equal halves, $S = S_0 \cup S_1$. We train $M$ models $\{h_{0,1}, \ldots, h_{0,M}\}$ on the first half of the data and validate them on the remaining half. Let $\hat{L}(h_{0,i}, S_1)$ for $i \in \{1, \ldots, M\}$ be the corresponding validation losses. Then we train another $M$ models $\{h_{1,1}, \ldots, h_{1,M}\}$ on the second half of the data and validate them on the first half. Let $\hat{L}(h_{1,i}, S_0)$ for $i \in \{1, \ldots, M\}$ be the corresponding validation losses. Finally, we select the model $h_{j^*,i^*} = \arg \min_{j \in \{0,1\}, i \in \{1,\ldots,M\}} \hat{L}(h_{j,i}, S_{1-j})$ with the smallest validation loss.

Derive a high-probability generalization bound for the expected loss of $h_{j^*,i^*}$. (I.e., a bound on $L(h_{j^*,i^*})$ that holds with probability at least $1 - \delta$.)

*Comment: no theorem in the lecture notes directly applies to the question, because they all assume that $\hat{L}(h, S)$ is computed on the same $S$ for all $h$. You have to make a custom derivation, but it will not be very different from derivations you can find in the lecture notes.*

# References


A. L. Goldberger, L. A. N. Amaral, L. Glass, J. M. Hausdorff, P. Ch. Ivanov, R. G. Mark, J. E. Mietus, G. B. Moody, C.-K. Peng, and H. E. Stanley. PhysioBank, PhysioToolkit, and PhysioNet: Components of a new research resource for complex physiologic signals. *Circulation*, 101(23):e215–e220, 2000.

C. Iber and AASM. *The AASM manual for the scoring of sleep and associated events: rules, terminology and technical specifications*. American Academy of Sleep Medicine, Westchester, I. L., 2007.

A. Kales and A. Rechtschaffen. *A manual of standardized terminology, techniques and scoring system for sleep stages of human subjects. Allan Rechtschaffen and Anthony Kales, editors*. U. S. National Institute of Neurological Diseases and Blindness, Neurological Information Network Bethesda, Md, 1968.

B. Kemp, A. H. Zwinderman, B. Tuk, H. A. C. Kamphuisen, and J. J. L. Oberye. Analysis of a sleep-dependent neuronal feedback loop: the slow-wave microcontinuity of the EEG. *IEEE Transactions on Biomedical Engineering*, 47(9): 1185–1194, 2000.

M. Perslev, M. Hejselbak Jensen, S. Darkner, P. J. Jennum, and C. Igel. U-time: A fully convolutional network for time series segmentation applied to sleep staging. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.